

Homework 2

19.5/20

Deadline: Wednesday 14 May 2024 (by 19h00)

Great work!

Credits: 20 points

Instructions:

- When you finish, please submit a single **.ipynb** file via email to wbanda@yachaytech.edu.ec
- The quiz is individual. Please include your name in the notebook.
- Within a **single python notebook**, solve the following problems.

Name:

Instructions:

- When you finish, please send me the **.ipynb** file via email to wbanda@yachaytech.edu.ec
- The exam is individual. Please include your name in the notebook.
- Within a **single python notebook**, solve the following problems:

In [40]:

```
# Import libraries
import numpy as np
import matplotlib.pyplot as plt
import pyvista as pv
import pandas as pd
import os
import cv2 as cv2
plt.style.use('dark_background')
from scipy.fftpack import fft2, fftshift, ifft2, ifftshift
from scipy.ndimage import gaussian_filter
import glob
from PIL import Image
import os
```

The Orszag-Tang vortex: Calculus and Fourier analysis (20 points)

We want to study the properties of turbulent flows using numerical calculus and Fourier analysis. Let us consider the Orszag-Tang vortex system, which describes a doubly periodic fluid configuration leading to 2D supersonic magnetohydrodynamical (MHD) turbulence. Although an analytical solution is not known, its simple and reproducible set of initial conditions has made it a widespread benchmark for the comparison of MHD numerical solvers.

The computational domain is periodic box with dimensions: $[0, 2\pi]^D$ where D is the number of spatial dimensions.

In code units, the initial conditions are given by:

$$\vec{v} = (-\sin y, \sin x), \quad \vec{B} = (-\sin y, \sin 2x), \quad \rho = 25/9, \quad p = 5/3$$

The numerical simulation produces 61 VTK files stored in:

- the **Orszag-Tang-MHD** folder:

https://github.com/wbandabarragan/physics-teaching-data/blob/main/2D-data/Orszag_Tang-MHD.zip

jointly with:

- a **units.out** file that contains the CGS normalisation values.
- a **vtk.out** file whose second column contains the times in code units.
- a **grid.out** file that contains information on the grid structure.

You can use VisIt to inspect the data. The written fields are:

- density (rho)
- thermal pressure (prs)
- velocity_x (vx1)
- velocity_y (vx2)
- magnetic_field_x (Bx1)
- magnetic_field_y (Bx2)

Reference paper: <https://arxiv.org/pdf/1001.2832.pdf>

"High-order conservative finite difference GLM-MHD schemes for cell-centered MHD", Mignone, Tzeferacos & Bodo, JCP (2010) 229, 5896.

4.75 / 5

1. (5 points) Numerical calculus:

(a) Create a set of Python functions that reads in the simulation data, normalises the data fields to CGS units (using units.out), returns all the data arrays (except magnetic fields, which won't be used) and the mesh, and sequentially prints the following figures into a folder called **"output_n"** for all times:

- Density, ρ
- Velocity curl (vorticity) magnitude, $|\vec{\nabla} \times \vec{v}|$

Add time-stamps in CGS units to the above maps (using information from vkt.out). Show sample maps at a fixed time (VTK file # 40) in this notebook.

(b) Create a set of Python functions that loops over all the simulation VTK files, computes the flow circulation, $\Gamma = \iint_S \vec{\nabla} \times \vec{v} \cdot d\mathbf{S}$, and returns:

- a CSV file with 2 columns (time and flow circulation).
- a figure of the flow circulation versus time. Show the resulting figure in this notebook too.

Dear Wladimir. I modified the Unzipped folder structure, so with the objective you dont have to move the images and things as I did, here are the changes i made in an automated code c:

```
In [41]: import zipfile
import os
import shutil  What is this library for?  Comment code.

# Define the path to the zip file and the target directory
zip_file_path = './Orszag_Tang-MHD.zip'
target_directory = './Orszag_Tang-MHD'

# Unzip the file
if not os.path.exists(target_directory):
    os.makedirs(target_directory, exist_ok=True)
    with zipfile.ZipFile(zip_file_path, 'r') as zip_ref:
        zip_ref.extractall(target_directory)
        print(f"Extracted contents of {zip_file_path} to {target_directory}")
else:
    print(f"Directory {target_directory} already exists. Skipping extraction")

# Move files from the inner folder to the main folder
inner_folder = os.path.join(target_directory, 'Orszag_Tang-MHD')
if os.path.exists(inner_folder):
    for item in os.listdir(inner_folder):
        source = os.path.join(inner_folder, item)
        destination = os.path.join(target_directory, item)
        shutil.move(source, destination)
    os.rmdir(inner_folder) # Remove the now-empty inner folder
    print(f"Moved contents of {inner_folder} to {target_directory}")
else:
    print(f"No inner folder named 'Orszag_Tang-MHD' found in {target_directory}")
```

Extracted contents of ./Orszag_Tang-MHD.zip to ./Orszag_Tang-MHD
 Moved contents of ./Orszag_Tang-MHD/Orszag_Tang-MHD to ./Orszag_Tang-MHD

Now, you can run the whole code c:

Sorry for the delay :c

```
In [42]: def units_data(fname):
        """
        Function to load and calculate normalization values from 'units.out'.
        """
        # Read the 'units.out' file
        df_units = pd.read_csv(fname + '/units.out', sep=',')

        # Extract normalization values into a list of tuples
        units = [(row['variable'], row['normalisation']) for _, row in df_units.iterrows()]

        # Calculate thermal pressure normalization
        pre_0 = units[0][1] * units[1][1]**2
        units.append(('pre_0', pre_0))

        # Calculate time normalization
        time_0 = units[2][1] / units[1][1]
        units.append(('time_0', time_0))

        # Return all normalization values
        return units

fname = './Orszag_Tang-MHD'
# Retrieve normalization values
units_cgs = units_data(fname)
print("Normalization values in CGS units:")
for unit in units_cgs:
    print(f"{unit[0]}: {unit[1]}")
```

Normalization values in CGS units:

Not needed. rho_0: 1.673e-24
v_0: 100000.0
L_0: 149600000000000.0
pre_0: 1.673e-14
time_0: 149600000.0

```
In [43]: def time_vtk(fname, units):
        """
        This function loads the time data from the file
        and returns it as a numpy array in CGS units.
        """
        # Read the 'vtk.out' file
        df_time = pd.read_csv(fname + '/vtk.out', sep=r'\s+', header=None)

        # Extract the second column (time data) and convert to numpy array
        time = np.array(df_time.iloc[:, 1], dtype=float)

        # Convert time to CGS units using the normalization factor
        time = time * units[4][1]

        return time

# Get the time data in CGS units
```

Comment out:

-0.25

```
time_cgs = time_vtk(fname, units_cgs)
```

```
# Print the time array  
print('Time:', time_cgs)
```

```
Time: [0.00000000e+00 1.48526306e+07 2.98050025e+07 4.45746814e+07  
5.92009088e+07 7.43835286e+07 8.95110806e+07 1.04652605e+08  
1.19212784e+08 1.34577707e+08 1.49056354e+08 1.64457823e+08  
1.79039634e+08 1.94200248e+08 2.09381955e+08 2.23998025e+08  
2.39347434e+08 2.54154243e+08 2.69057246e+08 2.84061976e+08  
2.99105602e+08 3.13576261e+08 3.28902482e+08 3.43798752e+08  
3.58774759e+08 3.73613882e+08 3.88617865e+08 4.03755889e+08  
4.18235822e+08 4.33358288e+08 4.48318886e+08 4.63722450e+08  
4.78257138e+08 4.93148172e+08 5.08328234e+08 5.23562450e+08  
5.38002142e+08 5.53433681e+08 5.68077875e+08 5.82903086e+08  
5.98031984e+08 6.13276822e+08 6.27824525e+08 6.42780785e+08  
6.57958453e+08 6.72861455e+08 6.88095522e+08 7.03095466e+08  
7.17794264e+08 7.32673929e+08 7.47686887e+08 7.62622502e+08  
7.77614666e+08 7.92765556e+08 8.07411546e+08 8.22541790e+08  
8.37337380e+08 8.52284215e+08 8.67354770e+08 8.82569239e+08  
8.97600000e+08]
```

```
In [44]: mesh = pv.read(fname + '/data.0000.vtk')  
print(mesh.array_names)
```

```
['rho', 'vx1', 'vx2', 'Bx1', 'Bx2', 'prs']
```

```
In [45]: # Function to load VTK data  
def vtk_data(fname, file_number):  
    """
```

```
    This function loads the vtk data from the file  
    and returns it as a numpy array  
    """
```

```
    # Retrieve normalization units and time data  
    units = units_data(fname)  
    time = time_vtk(fname, units)
```

```
    # Read the VTK file  
    mesh = pv.read(fname + f'/data.{file_number:04d}.vtk')
```

```
    # Extract and normalize data fields  
    rho = pv.get_array(mesh, 'rho', preference='cell') * units[0][1]  
    vx1 = pv.get_array(mesh, 'vx1', preference='cell') * units[1][1]  
    vx2 = pv.get_array(mesh, 'vx2', preference='cell') * units[1][1]  
    prs = pv.get_array(mesh, 'prs', preference='cell') * units[3][1]
```

```
    # Reshape data to 2D arrays based on mesh dimensions  
    dimensions = mesh.GetDimensions()  
    rho_cgs2D = rho.reshape(dimensions[1] - 1, dimensions[0] - 1)  
    vx1_cgs2D = vx1.reshape(dimensions[1] - 1, dimensions[0] - 1)  
    vx2_cgs2D = vx2.reshape(dimensions[1] - 1, dimensions[0] - 1)  
    prs_cgs2D = prs.reshape(dimensions[1] - 1, dimensions[0] - 1)
```

```
    # Create 2D mesh grid for coordinates  
    x = np.linspace(mesh.bounds[0], mesh.bounds[1], dimensions[0] - 1) * uni
```



```
y = np.linspace(mesh.bounds[2], mesh.bounds[3], dimensions[1] - 1) * uni
x2D, y2D = np.meshgrid(x, y)

# Pack data and mesh into tuples
data = (rho_cgs2D, vx1_cgs2D, vx2_cgs2D, prs_cgs2D)
mesh = (x2D, y2D)
vtk_data = (mesh, data, time)

return vtk_data
```

In [46]:



```
# Load VTK data
(mesh, data, time) = vtk_data(fname, 40)
x2D, y2D = mesh
rho, vx1, vx2, prs = data
```



```
# Get 1D coordinate arrays
x_coords = x2D[0, :]
y_coords = y2D[:, 0]
```



```
# Compute gradients for vorticity
dvx_dy, dvx_dx = np.gradient(vx1, y_coords, x_coords)
dvy_dy, dvy_dx = np.gradient(vx2, y_coords, x_coords)
```



```
# Compute vorticity magnitude
vorticity = np.abs(dvy_dx - dvx_dy)

fig, ax = plt.subplots(1, 2, figsize=(14, 7))
```

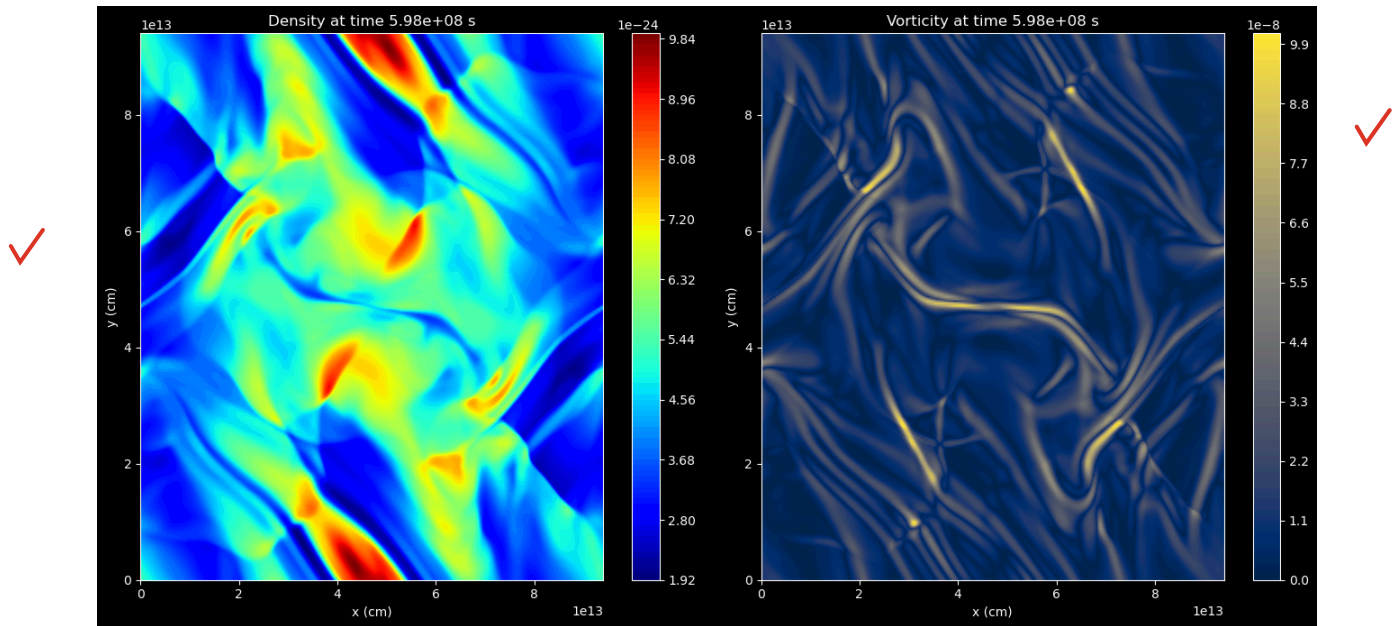


```
# Plot density
z1 = ax[0].contourf(x2D, y2D, rho, levels=100, cmap='jet', vmin=rho.min(), v
fig.colorbar(z1, ax=ax[0])
ax[0].set_title(f'Density at time {time[40]:.2e} s')
ax[0].set_xlabel('x (cm)')
ax[0].set_ylabel('y (cm)')
```



```
# Plot vorticity
z2 = ax[1].contourf(x2D, y2D, vorticity, levels=100, cmap='cividis', vmin=vc
fig.colorbar(z2, ax=ax[1])
ax[1].set_title(f'Vorticity at time {time[40]:.2e} s')
ax[1].set_xlabel('x (cm)')
ax[1].set_ylabel('y (cm)')

plt.tight_layout()
```



```
In [47]: target = './output_n'
# Create the output directory if it doesn't exist
os.makedirs(target, exist_ok=True)

# Function to compute vorticity
def compute_vorticity(vx1, vx2, x_coors, y_coors):
    """
    Compute the vorticity magnitude from velocity components.

    Args:
        vx1 (numpy.ndarray): 2D array of the x-component of the velocity field
        vx2 (numpy.ndarray): 2D array of the y-component of the velocity field
        x_coors (numpy.ndarray): 1D array of x-coordinates.
        y_coors (numpy.ndarray): 1D array of y-coordinates.

    Returns:
        numpy.ndarray: 2D array of the vorticity magnitude.
    """
    # Compute gradients of the velocity components
    dvx_dy, dvx_dx = np.gradient(vx1, y_coors, x_coors)
    dvy_dy, dvy_dx = np.gradient(vx2, y_coors, x_coors)

    # Compute vorticity as the difference of partial derivatives
    vorticity = np.abs(dvy_dx - dvx_dy)
    return vorticity

# Loop over all VTK files
for i in range(61):
    try:
        # Load VTK data for the current file
        vtk_result = vtk_data(fname, i)
        mesh, data, time = vtk_result
        rho, vx1, vx2, _ = data

        # Get 1D coordinate arrays
        x2D, y2D = mesh
        x_coors = x2D[0, :]
```

```

y_coords = y2D[:, 0]

# Compute vorticity
vorticity = compute_vorticity(vx1, vx2, x_coords, y_coords)

# Create a multiplot figure
fig, ax = plt.subplots(1, 2, figsize=(14, 7))

# Plot density
z1 = ax[0].contourf(x2D, y2D, rho, levels=100, cmap='jet', vmin=rho.
fig.colorbar(z1, ax=ax[0])
ax[0].set_title(f'Density at time {time[i]:.2e} s')
ax[0].set_xlabel('x (cm)')
ax[0].set_ylabel('y (cm)')

# Plot vorticity
z2 = ax[1].contourf(x2D, y2D, vorticity, levels=100, cmap='magma', v
fig.colorbar(z2, ax=ax[1])
ax[1].set_title(f'Vorticity at time {time[i]:.2e} s')
ax[1].set_xlabel('x (cm)')
ax[1].set_ylabel('y (cm)')

# Save the multiplot
plt.tight_layout()
plt.savefig(f'{target}/multiplot_{i:04d}.png')
plt.close()

except Exception as e:
    print(f"Error processing file {i}: {e}")

```

```

In [48]: target_csv = './flow_circulation.csv'
# Initialize an empty list to store time and circulation values
circulation_data = []

# Loop over all 61 VTK files (from 0 to 60)
for i in range(61):
    try:
        # Load VTK data
        vtk_result = vtk_data(fname, i)
        mesh, data, time = vtk_result
        _, vx1, vx2, _ = data

        # Get 1D coordinate arrays
        x2D, y2D = mesh
        x_coords = x2D[0, :]
        y_coords = y2D[:, 0]

        # Compute gradients for vorticity
        dvx_dy, dvx_dx = np.gradient(vx1, y_coords, x_coords)
        dvy_dy, dvy_dx = np.gradient(vx2, y_coords, x_coords)

        # Compute vorticity magnitude
        vorticity = dvy_dx - dvx_dy

        # Compute flow circulation as the integral of vorticity over the dom
        dx = np.diff(x_coords).mean()

```

✓

```

dy = np.diff(y_coords).mean()
circulation = np.sum(vorticity) * dx * dy

# Append time and circulation to the list
circulation_data.append((time[i], circulation))

except Exception as e:
    print(f"Error processing file {i}: {e}")

```

✓

```

# Save the circulation data to a CSV file
df_circulation = pd.DataFrame(circulation_data, columns=["Time_(s)", "Flow_Circulation"])
df_circulation.to_csv(target_csv, index=False)
print(f"Flow circulation data saved to {target_csv}")

```

Flow circulation data saved to ./flow_circulation.csv

In [49]:

✓

```

# Load the CSV file
df = pd.read_csv(target_csv)

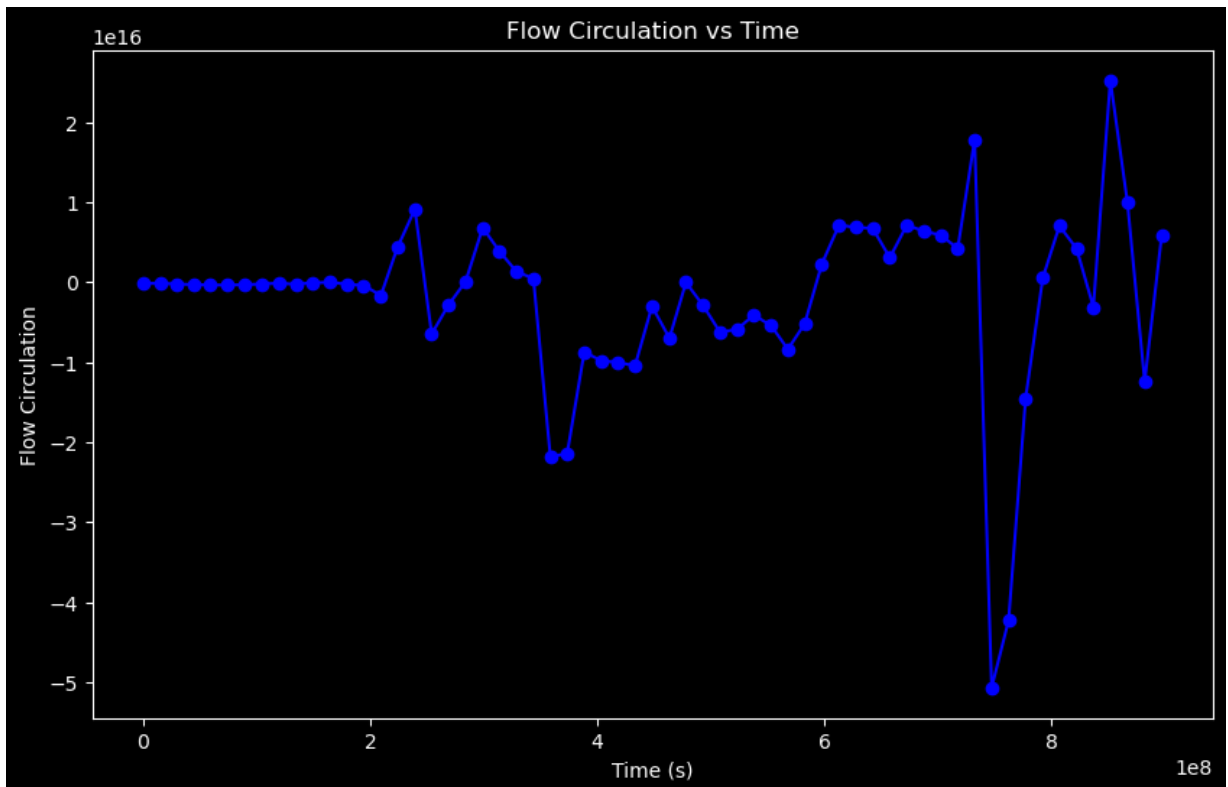
```

✓

```

# Plot the flow circulation versus time
plt.figure(figsize=(10, 6))
plt.plot(df["Time_(s)"], df["Flow_Circulation"], marker='o', linestyle='-',)
plt.title("Flow Circulation vs Time")
plt.xlabel("Time (s)")
plt.ylabel("Flow Circulation")
plt.show()

```



5 / 5

2. (5 points) Shock analysis:

(c) Create a set of Python functions that isolate candidate shocked cells based on the following methods and sequentially prints the following shock candidate maps into a

folder called "**output_s**" for all times:

- **Method 1:** Read the 2D velocity vector field. Compute the divergence of the velocity field and isolate the cells where there are convergent flows (i.e. where $\vec{\nabla} \cdot \vec{v} < \alpha$ with $\alpha \lesssim 0$). Cells with convergent flows are candidate shocked cells.
- **Method 2:** Read the 2D pressure field. Compute the gradient of the pressure and isolate the cells with large pressure gradients (i.e. where $\frac{|\vec{\nabla} P|}{P} > \beta \max \left(\frac{|\vec{\nabla} P|}{P} \right)$ with $\beta \lesssim 0.1$). Such cells are candidate shocked cells.

(d) Make binary maps of the resulting candidate shock cells from both methods. Show sample maps at a fixed time (VTK file # 40) in this notebook.

```
In [50]: def method1_convergent_flows(vx1, d_spacing, alpha):
        """
        Identify cells with convergent flows based on the velocity divergence.

        Args:
            vx1 (numpy.ndarray): 2D array of the x-component of the velocity field.
            d_spacing (float): Grid spacing in the spatial domain.
            alpha (float): Threshold for identifying convergent flows. Cells with
                               divergence less than alpha are considered convergent.

        Returns:
            numpy.ndarray: Boolean 2D array where True indicates cells with convergent flows.
        """
        grad_v = np.gradient(vx1, d_spacing)
        divergence = grad_v[0] + grad_v[1]
        return divergence < alpha

# Method 2: Identify cells with large pressure gradients
def method2_large_pressure_gradients(prs, d_spacing, beta):
    """
    Identify cells with large pressure gradients.

    Args:
        prs (numpy.ndarray): 2D array of the pressure field.
        d_spacing (float): Grid spacing in the spatial domain.
        beta (float): Threshold for identifying large pressure gradients. Cells with
                               normalized gradient greater than beta * max are considered shocked.

    Returns:
        numpy.ndarray: Boolean 2D array where True indicates cells with large pressure gradients.
    """
    prs_grad = np.gradient(prs, d_spacing)
    lhs = np.abs(prs_grad[0]) / prs
    max_nabla_prs = np.max(lhs)
    return lhs > beta * max_nabla_prs

# Define output folder for shock candidate maps
target_shock = './output_s'
os.makedirs(target_shock, exist_ok=True)

# Loop over all 61 VTK files
for i in range(61):
```

```

try:
    # Load VTK data
    vtk_result = vtk_data(fname, i)
    mesh, data, time = vtk_result
    _, vx1, vx2, prs = data

    # Get 1D coordinate arrays
    x2D, y2D = mesh
    x_coords = x2D[0, :]
    y_coords = y2D[:, 0]
    d_spacing = np.diff(x_coords).mean()

    # Apply Method 1
    method1 = method1_convergent_flows(vx1, d_spacing, alpha=-1e-9)

    # Apply Method 2
    method2 = method2_large_pressure_gradients(prs, d_spacing, beta=0.1)

    # Create binary maps for both methods
    fig, ax = plt.subplots(1, 2, figsize=(14, 7))

    # Plot Method 1 binary map
    im1 = ax[0].imshow(method1, extent=[x_coords.min(), x_coords.max(),
                                       origin='lower', cmap='binary')
    ax[0].set_title(f'Method 1: Convergent Flows (Time {time[i]:.2e} s),
    ax[0].set_xlabel('x (cm)')
    ax[0].set_ylabel('y (cm)')

    # Plot Method 2 binary map
    im2 = ax[1].imshow(method2, extent=[x_coords.min(), x_coords.max(),
                                       origin='lower', cmap='binary')
    ax[1].set_title(f'Method 2: Large Pressure Gradients (Time {time[i]:
    ax[1].set_xlabel('x (cm)')
    ax[1].set_ylabel('y (cm)')

    # Add a global colorbar
    cbar = fig.colorbar(im1, ax=ax, orientation='vertical', fraction=0.0
    cbar.set_label('Shock Candidate')

    # Save the binary maps
    plt.savefig(f'{target_shock}/shock_candidates_{i:04d}.png')
    plt.close()

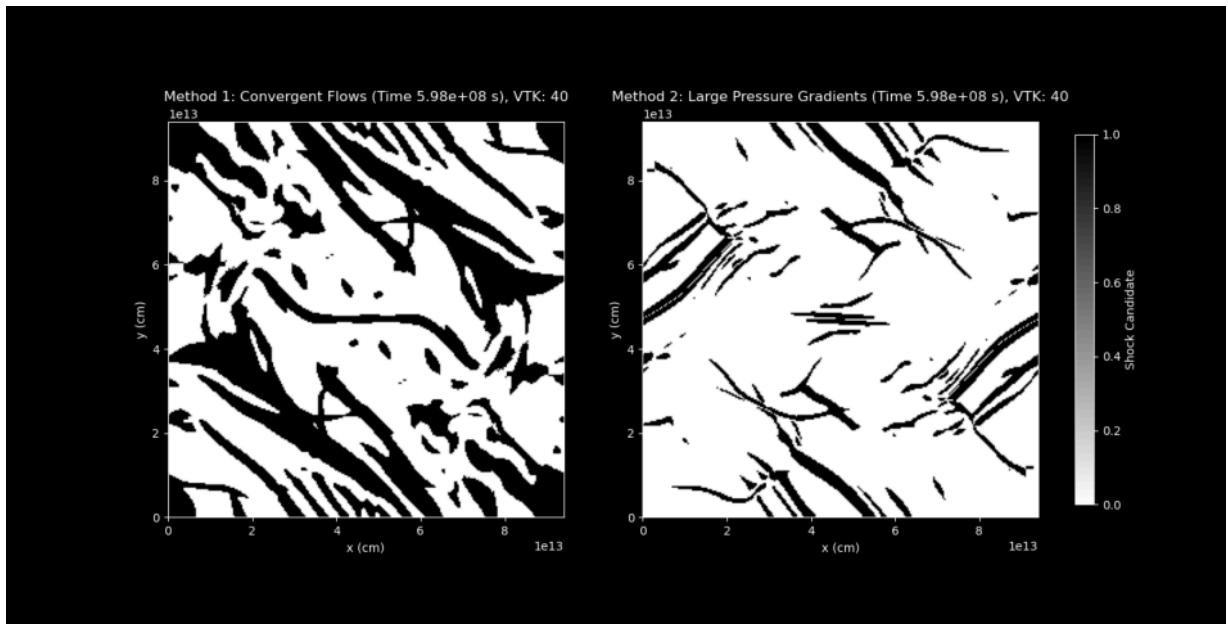
except Exception as e:
    print(f"Error processing file {i}: {e}")

```

```

In [51]: # Load and display a sample shock candidate map
sample_shock_map = plt.imread(f'{target_shock}/shock_candidates_0040.png')
plt.figure(figsize=(12,12))
plt.imshow(sample_shock_map, cmap='binary')
plt.axis('off')
plt.show()

```



4.75/5

3. (5 points) Fourier analysis:

(e) Create a set of Python functions that Fourier transforms the density of any VTK file in 2D, applies high-pass and low-pass filters, smooths the density map using a 2D Gaussian, and sequentially prints the following figures into a folder called **"output_f"** for all times:

- the 2D Fourier image of the density
- the high-pass filter map
- the low-pass filter map
- the Gaussian-blurred map

Show sample maps at a fixed time (VTK file # 40) in this notebook.

(f) Create a Python function that reads in the images you wrote, and returns movies showing the time evolution of the velocity curl maps computed in (a), the high-pass filter maps in (e), and the flow circulation in (b).

```
In [52]: def high_pass_filter(spectrum, threshold=0.05):
        """
        Apply high-pass filter to frequency domain data.

        Args:
            spectrum_data: Fourier-transformed data (shifted)
            threshold: Threshold for frequency cutoff

        Returns:
            Filtered data in spatial domain
        """
```

```

✓ rows, cols = spectrum.shape
  center_row, center_col = rows // 2, cols // 2

  # Create high-pass mask (blocks low frequencies)
✓ mask = np.ones_like(spectrum, dtype=bool)
  radius = int(min(rows, cols) * threshold)

  # Create circular mask
✓ y2D, x2D = np.meshgrid(np.arange(rows), np.arange(cols), indexing='ij')
  distance = np.sqrt((y2D - center_row)**2 + (x2D - center_col)**2)
  mask[distance < radius] = 0

  # Apply filter
✓ filtered_spectrum = spectrum * mask

  # Transform back to spatial domain
✓ return np.abs(fft2(fftshift(filtered_spectrum)))

def low_pass_filter(spectrum, threshold=0.05):
    """
    Apply low-pass filter to frequency domain data.

    Args:
        spectrum_data: Fourier-transformed data (shifted)
        threshold: Threshold for frequency cutoff

    Returns:
        Filtered data in spatial domain
    """
    rows, cols = spectrum.shape
    center_row, center_col = rows // 2, cols // 2

    # Create low-pass mask (keeps only low frequencies)
✓ mask = np.zeros_like(spectrum, dtype=bool)
    radius = int(min(rows, cols) * threshold)

    # Create circular mask
✓ y2D, x2D = np.meshgrid(np.arange(rows), np.arange(cols), indexing='ij')
    distance = np.sqrt((y2D - center_row)**2 + (x2D - center_col)**2)
    mask[distance < radius] = 1

    # Apply filter
✓ filtered_spectrum = spectrum * mask

    # Transform back to spatial domain
✓ return np.abs(fft2(fftshift(filtered_spectrum)))

# Define output folder for Fourier analysis
✓ target_fourier = './output_f'
  os.makedirs(target_fourier, exist_ok=True)

  for i in range(61):
      try:
          # Load VTK data
          ✓ vtk_result = vtk_data(fname, i)
            mesh, data, time = vtk_result

```

```

rho, _, _, _ = data

# Get coordinate information
x2D, y2D = mesh
x_coords = x2D[0, :]
y_coords = y2D[:, 0]

# Perform 2D Fourier transform
fft_rho = fft2(rho)
fft_rho_shifted = fftshift(fft_rho)

# Apply filters
high_pass_map = high_pass_filter(fft_rho_shifted, threshold=0.05)
low_pass_map = low_pass_filter(fft_rho_shifted, threshold=0.10)
gaussian_map = gaussian_filter(rho, sigma=5)

# Create a multiplot figure
fig, ax = plt.subplots(2, 2, figsize=(14, 14))

# Plot 2D Fourier image
im1 = ax[0, 0].imshow(np.log(np.abs(fft_rho_shifted)), cmap='viridis')
ax[0, 0].set_title('2D Fourier Image')
ax[0, 0].axis('off')
cbar1 = fig.colorbar(im1, ax=ax[0, 0], orientation='vertical', fraction=0.05)
cbar1.set_label('Log Intensity')

# Plot high-pass filter map
im2 = ax[0, 1].imshow(high_pass_map, cmap='plasma')
ax[0, 1].set_title('High-Pass Filter Map')
ax[0, 1].axis('off')
cbar2 = fig.colorbar(im2, ax=ax[0, 1], orientation='vertical', fraction=0.05)
cbar2.set_label('Intensity')

# Plot low-pass filter map
im3 = ax[1, 0].imshow(low_pass_map, cmap='cividis')
ax[1, 0].set_title('Low-Pass Filter Map')
ax[1, 0].axis('off')
cbar3 = fig.colorbar(im3, ax=ax[1, 0], orientation='vertical', fraction=0.05)
cbar3.set_label('Intensity')

# Plot Gaussian-blurred map
im4 = ax[1, 1].imshow(gaussian_map, cmap='magma')
ax[1, 1].set_title('Gaussian-Blurred Map')
ax[1, 1].axis('off')
cbar4 = fig.colorbar(im4, ax=ax[1, 1], orientation='vertical', fraction=0.05)
cbar4.set_label('Intensity')

# Save the multiplot
plt.tight_layout()
plt.savefig(f'{target_fourier}/fourier_analysis_{i:04d}.png')
plt.close()

except Exception as e:
    print(f"Error processing file {i}: {e}")

# Display a sample map for VTK file #40

```

✓

```
sample_fourier_map = plt.imread(f'{target_fourier}/fourier_analysis_0040.png')
plt.figure(figsize=(12,12))
plt.imshow(sample_fourier_map)
plt.axis('off')
plt.show()
```

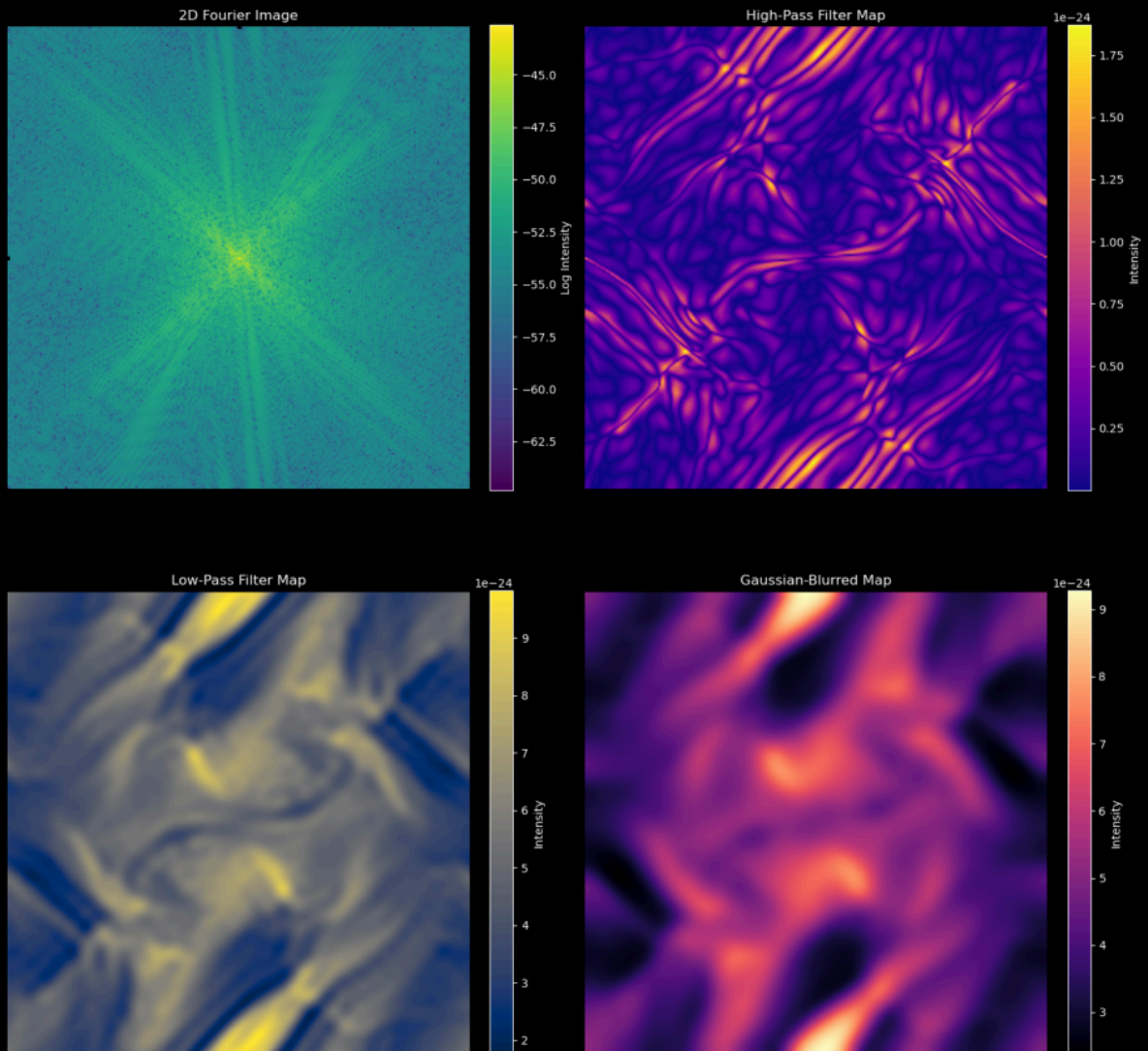
```
/tmp/ipykernel_53374/6307807.py:88: RuntimeWarning: divide by zero encountered in log
```

```
im1 = ax[0, 0].imshow(np.log(np.abs(fft_rho_shifted)), cmap='viridis')
```

```
/tmp/ipykernel_53374/6307807.py:88: RuntimeWarning: divide by zero encountered in log
```

```
im1 = ax[0, 0].imshow(np.log(np.abs(fft_rho_shifted)), cmap='viridis')
```

✓



✓

✓

✓

In [53]:

✓

```
# Get all folders starting with "output_" in the current working directory
output_folders = [folder for folder in glob.glob('./output_*') if os.path.is
```

```
# Iterate over each folder
```

✓

```
for folder in output_folders:
```

```
# Extract the folder name to use as the GIF name
```

```
folder_name = os.path.basename(folder)
```

✓

```
# Define the keyword for image selection
keyword = "multiplot" if "n" in folder_name else "shock_candidates" if "
```

✓

```
# Define the input image pattern and output GIF path
images_input = os.path.join(folder, f"{keyword}*.png")
gif_output = os.path.join('./', f"{keyword}.gif")
```

✓

```
# Collect the images
imgs = (Image.open(f) for f in sorted(glob.glob(images_input)))
img = next(imgs, None)
```

✓

```
# If there are images, create the GIF
if img:
    img.save(fp=gif_output, format="GIF", append_images=imgs, save_all=True)
    print(f"GIF created: {gif_output}")
```

✓

```
else:
    print(f"No images found in {folder} for keyword '{keyword}'")
```

✓

```
GIF created: ./multiplot.gif
GIF created: ./shock_candidates.gif
GIF created: ./fourier_analysis.gif
```

In [54]: `from IPython.display import Image as IPyImage`

-0.25

```
# display(IPyImage(filename='./multiplot.gif'))
# display(IPyImage(filename='./shock_candidates.gif'))
# display(IPyImage(filename='./fourier_analysis.gif'))
```

Movies look ok, but the colour bar limits should be set and enforced to compare between times.

5 / 5

Excellent
analysis!

4. (5 points) Interpretation

(g) Based on your analyses above, briefly answer the following questions:

- What information do the velocity divergence and velocity curl provide about the flow in Section 1?

The curl of the velocity field, also known as the **vorticity**, characterizes the local rotational behavior of the fluid. ✓ It gives a measure of how much a small fluid element is spinning, and it plays a key role in identifying rotational structures such as vortices. ✓ Monitoring vorticity helps reveal the development of instabilities and the onset of turbulent behavior, especially in regions affected by shock waves or sudden changes in flow velocity. ✓

The divergence of the velocity field quantifies how much the fluid is expanding (positive divergence) ✓ or compressing (negative divergence) ✓ at a given point. In the context of nearly incompressible fluids, significant negative divergence values can signal the presence of shock fronts or compressive disturbances. ✓ This quantity was particularly useful in the analysis as a diagnostic tool to locate potential shock wave regions, since those features tend to produce sharp compressive gradients that manifest as pronounced negative divergence. ✓

- **Does the flow circulation reach steady state in Section 1?**

No, it does not. The time evolution of the flow circulation indicates that the system does not settle into a steady or laminar configuration. Around $t = 2 \cdot 10^8$, the circulation begins to exhibit more erratic and irregular behavior, which suggests a transition to **turbulence**. In this regime, the fluid motion becomes chaotic, with persistent generation of vortices and complex interactions across scales.

- **Do you find the same shock candidates on the maps obtained by Methods 1 and 2 in Section 2?**

Not exactly. While both methods generally highlight similar regions as potential shock locations, there are noticeable differences in specific cases. In some of the visualizations, the methods identify significantly different cells as shock candidates. These discrepancies might arise due to variations in sensitivity, resolution, or the criteria used by each method to detect compressive features.

- **What do the high-pass and low-pass filter maps show in Section 3?**

The **low-pass filter** emphasizes the **large-scale structures** of the flow by smoothing out high-frequency components. It reveals the overall motion and broad flow patterns of the liquid, such as large vortices or bulk displacement.

In contrast, the **high-pass filter** isolates the **small-scale features** by removing the background structure and highlighting sharp gradients. These details include localized compressions and discontinuities, which are typically associated with shock waves or rapid flow transitions.

- **Are the low-pass and Gaussian blurring results consistent with one another in Section 3?**

Yes, but the degree of similarity depends on the shape and size of the mask applied in the Fourier domain during the low-pass filtering. If the mask is relatively wide and smooth—resembling a Gaussian function—the result closely matches what is obtained via Gaussian blurring. Both techniques suppress high-frequency components and retain the large-scale flow features.

However, if the low-pass mask is narrow or sharply defined, the outcome diverges from the Gaussian case. In such scenarios, the low-pass filter can preserve more of the coarse structures but lose some of the transitional smoothness provided by Gaussian blurring.